

UNIVERSIDAD CATÓLICA DE CUENCA



**FACULTAD DE INFORMÁTICA, CIENCIAS DE LA COMPUTACIÓN E
INNOVACIÓN TECNOLÓGICA**

CARRERA DE ROBÓTICA E INTELIGENCIA ARTIFICIAL

ASIGNATURA: MICROCONTROLADORES

**TEMA: SISTEMA DE CONTROL DE TIEMPO REAL PARA CÉLULA
ROBOTICA**

NOMBRE DEL ESTUDIANTE:

Mateo Roca

DOCENTE: ING. PABLO BUESTÁN ANDRADE, PhD

FECHA: 3/05/2026

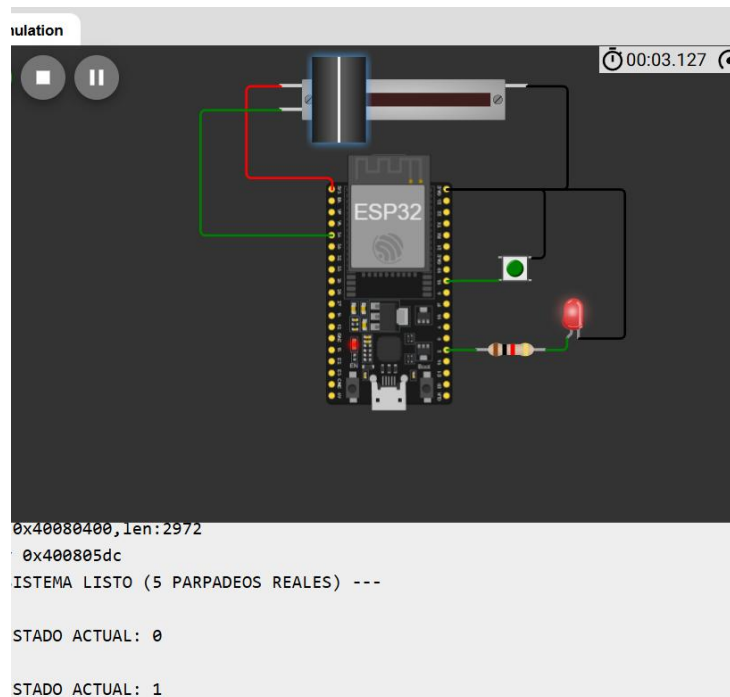
INTRODUCCIÓN

La integración de entradas analógicas y digitales bajo un esquema de interrupciones es fundamental para el desarrollo de sistemas embebidos de alta eficiencia. La eliminación de funciones bloqueantes como `delay()` permite que el microcontrolador mantenga una alta disponibilidad para tareas críticas. En este laboratorio, se implementa una máquina de estados que integra la lectura de un potenciómetro y una secuencia de parpadeo controlada por un temporizador de hardware.

OBJETIVO GENERAL

Implementar una máquina de estados finita en un ESP32 que gestione entradas analógicas y secuencias temporizadas mediante interrupciones de hardware, asegurando la transición automática entre estados de operación y reposo.

DISEÑO ESQUEMATICO



DESARROLLO

Configuración de interrupciones externas

Para lograr un sistema de control adaptativo, se implementó una rutina de servicio de interrupción (ISR) en el **GPIO 18**. Esta configuración permite que el microcontrolador detecte el cambio de flanco (FALLING) del pulsante de manera inmediata, eliminando la necesidad de consultar el estado del pin en el ciclo principal. Se integró un filtrado de rebotes mecánicos (Debounce) mediante software, asegurando que cada pulsación corresponda a una única transición de estado en la FSM.

Gestión determinista con hardware Timer

La precisión de la secuencia de alerta se garantiza mediante el uso de un **Hardware Timer** configurado a una frecuencia de **1 Hz** (un segundo por pulso). A diferencia de los métodos bloqueantes, esta técnica permite que la conmutación del LED y el conteo de los 5 ciclos de

parpadeo ocurran de forma asíncrona. Esto asegura que la base de tiempo sea constante y no se vea afectada por el procesamiento de otras tareas, como la adquisición de datos analógicos.

Lógica de máquina de estados

El funcionamiento del sistema se segmentó en tres fases de ejecución controladas por una unidad de acción central:

1. **Estado 0 (Standby):** Es el modo de reposo donde el actuador (LED) se mantiene en un nivel lógico bajo constante, representando un estado de inactividad total del sistema.
2. **Estado 1 (Control Analógico):** Se activa el procesamiento de señal en tiempo real. El sistema captura la entrada del potenciómetro (ADC de 12 bits) y realiza un mapeo determinista para transformarla en una señal de salida PWM de 8 bits, regulando la intensidad lumínica de forma fluida.
3. **Estado 2 (Secuencia de Alerta):** El control es asumido por la ISR del Timer. El sistema ejecuta exactamente 5 parpadeos y posee la "conciencia" necesaria para realizar un retorno automático a la condición de reposo tras completar la tarea.

CODIGO

```
#include <Arduino.h>

const int pinPot = 34;
const int pinBtn = 18;
const int pinLED = 2;

volatile int estado = 0;
volatile int parpadeosCount = 0;
volatile bool ledStatus = false;

hw_timer_t *timer = NULL;

void IRAM_ATTR isrBoton() {
    static uint32_t last_time = 0;
    uint32_t now = millis();
    if (now - last_time > 350) {
        estado++;
        if (estado > 2) estado = 0;

        if (estado == 2) {
            parpadeosCount = 0;
            ledStatus = false; // Empezamos apagado para que el primer cambio sea a encendido
        }
        last_time = now;
    }
}
```

```

}

void IRAM_ATTR onTimer() {
    if (estado == 2) {
        ledStatus = !ledStatus;
        digitalWrite(pinLED, ledStatus);

        // Contamos cada vez que el LED se APAGA
        if (ledStatus == false) {
            parpadeosCount++;
            // Ajuste: Cambiamos a 5 para asegurar que el quinto ciclo termine
            if (parpadeosCount >= 5) {
                estado = 0;
                parpadeosCount = 0;
            }
        }
    }
}

void setup() {
    Serial.begin(115200);
    pinMode(pinLED, OUTPUT);
    pinMode(pinBtn, INPUT_PULLUP);

    attachInterrupt(digitalPinToInterrupt(pinBtn), isrBoton, FALLING);

    timer = timerBegin(1000000);
    timerAttachInterrupt(timer, &onTimer);
    timerAlarm(timer, 1000000, true, 0);

    Serial.println("--- SISTEMA LISTO (5 PARPADEOS REALES) ---");
}

void loop() {
    static int estadoAnterior = -1;

    if (estado != estadoAnterior) {
        Serial.printf("\n>>> ESTADO ACTUAL: %d\n", estado);
        estadoAnterior = estado;

        // Forzamos el modo del pin para limpiar el PWM del estado 1
        pinMode(pinLED, OUTPUT);
        if(estado == 0) digitalWrite(pinLED, LOW);
    }
}

```

```

switch (estado) {
  case 0:
    digitalWrite(pinLED, LOW);
    break;

  case 1:
    analogWrite(pinLED, map(analogRead(pinPot), 0, 4095, 0, 255));
    break;

  case 2:
    // El parpadeo lo hace la ISR onTimer
    break;
}
}

```

ANALISIS DE RESULTADOS

- **Estabilidad de la FSM:** Se verificó mediante el Monitor Serial que las transiciones entre los estados 0, 1 y 2 son estables y responden instantáneamente a la interrupción externa sin bloqueos del procesador.
- **Precisión de Salida:** En el modo analógico, el escalamiento de la señal permitió una variación de brillo proporcional al giro del potenciómetro, validando la precisión del mapeo lineal de 4095 a 255 unidades.
- **Autogestión de Ciclos:** El sistema demostró capacidad de autogestión en el Estado 2, donde el contador de parpadeos activó la transición automática al Estado 0 de manera exitosa tras el quinto ciclo, cumpliendo con los requisitos de diseño asíncrono.

CONCLUSIONES

- Se implementó con éxito una arquitectura de control que prioriza la eficiencia del procesador al eliminar funciones bloqueantes, permitiendo una ejecución multitarea real en el ESP32.
- La combinación de interrupciones externas y temporizadores de hardware resultó ser la solución óptima para sistemas que requieren alta disponibilidad y precisión temporal en entornos robóticos.
- La lógica de estados finitos facilitó un flujo de trabajo organizado, asegurando que el sistema retorne a un estado seguro (Standby) de forma autónoma una vez finalizada una secuencia de alerta.

FOTOGRAFIAS

```
Salida - Monitor Serie X
Mensaje (Intro para mandar el mensaje de 'ESP32 Dev Module' a 'COM4')
Nueva línea
115200 baudios

>>> ESTADO ACTUAL: 0
>>> ESTADO ACTUAL: 1
>>> ESTADO ACTUAL: 2
>>> ESTADO ACTUAL: 0
>>> ESTADO ACTUAL: 1
```

